

工程造价智能编制及分析系统 - 详细设计说明书

文档信息

项目	内容
文档名称	工程造价智能编制及分析系统 - 详细设计说明书
版本号	v1.0
编写日期	2025年8月
编写人	系统开发团队
审核人	-
文档状态	正式版

目录

- [概述](#)
- [系统架构设计](#)
- [核心模块设计](#)
- [数据模型设计](#)
- [API接口设计](#)
- [任务处理流程设计](#)
- [关键技术实现](#)
- [部署与配置](#)
- [性能与安全](#)
- [扩展性设计](#)

1. 概述

1.1 系统简介

Excel工程数量表智能定额映射系统（以下简称"xparser"）是一款专为工程造价工程师设计的智能工具，能够自动将Excel格式的工程数量表中的工程项目映射到对应的标准定额编号，实现工程量的自动换算和造价计算。

1.2 核心功能

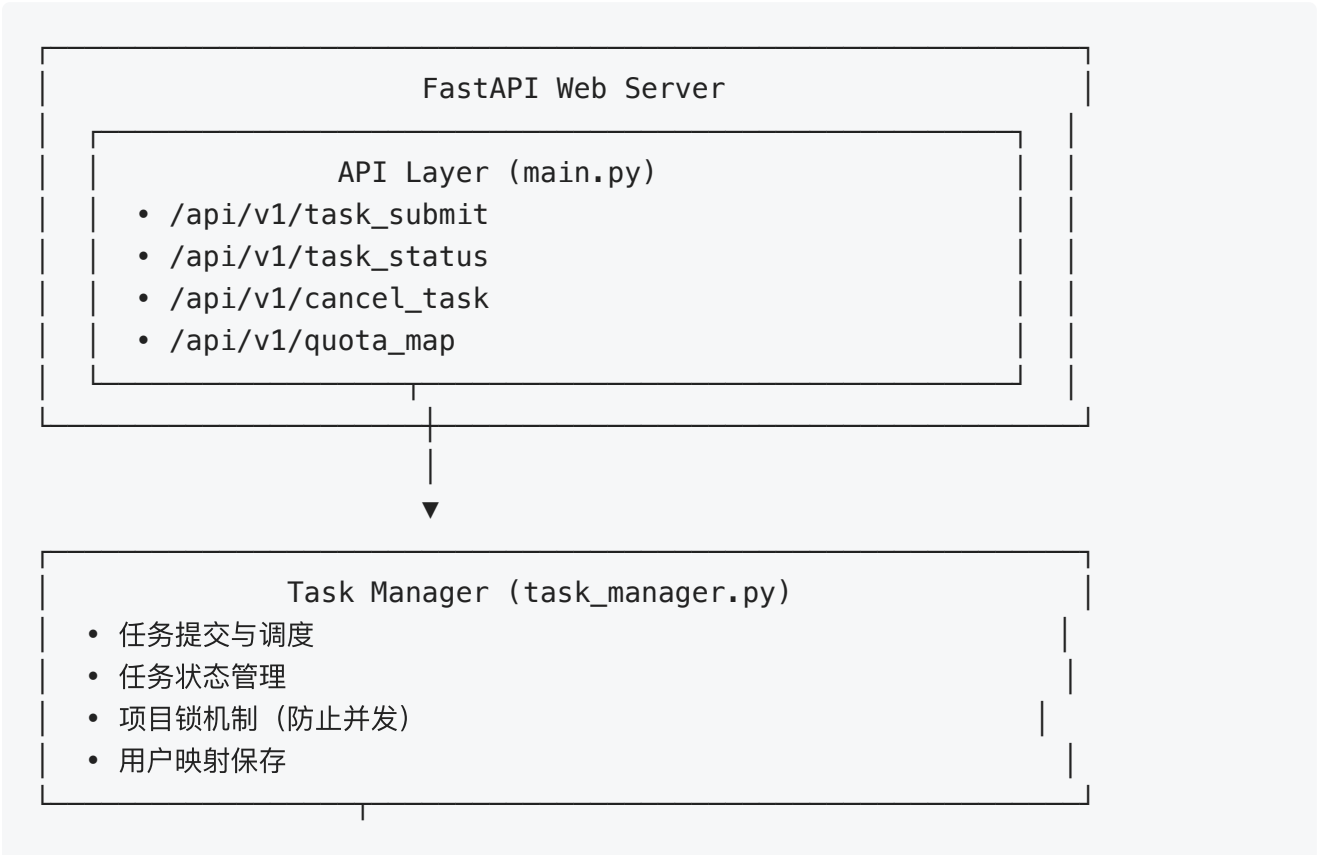
- **智能定额匹配**：基于向量相似度检索，自动识别Excel中的工程项目并匹配对应的定额编号
- **单位自动换算**：支持常见工程单位的自动识别和换算（米、平方米、立方米、吨等）
- **多模式处理**：支持普通模式、匹配模式、模板模式、对比模式四种处理方式
- **用户映射学习**：记录用户确认的映射关系，逐步优化匹配准确度
- **增量更新**：支持Excel文件版本对比，仅处理变化部分

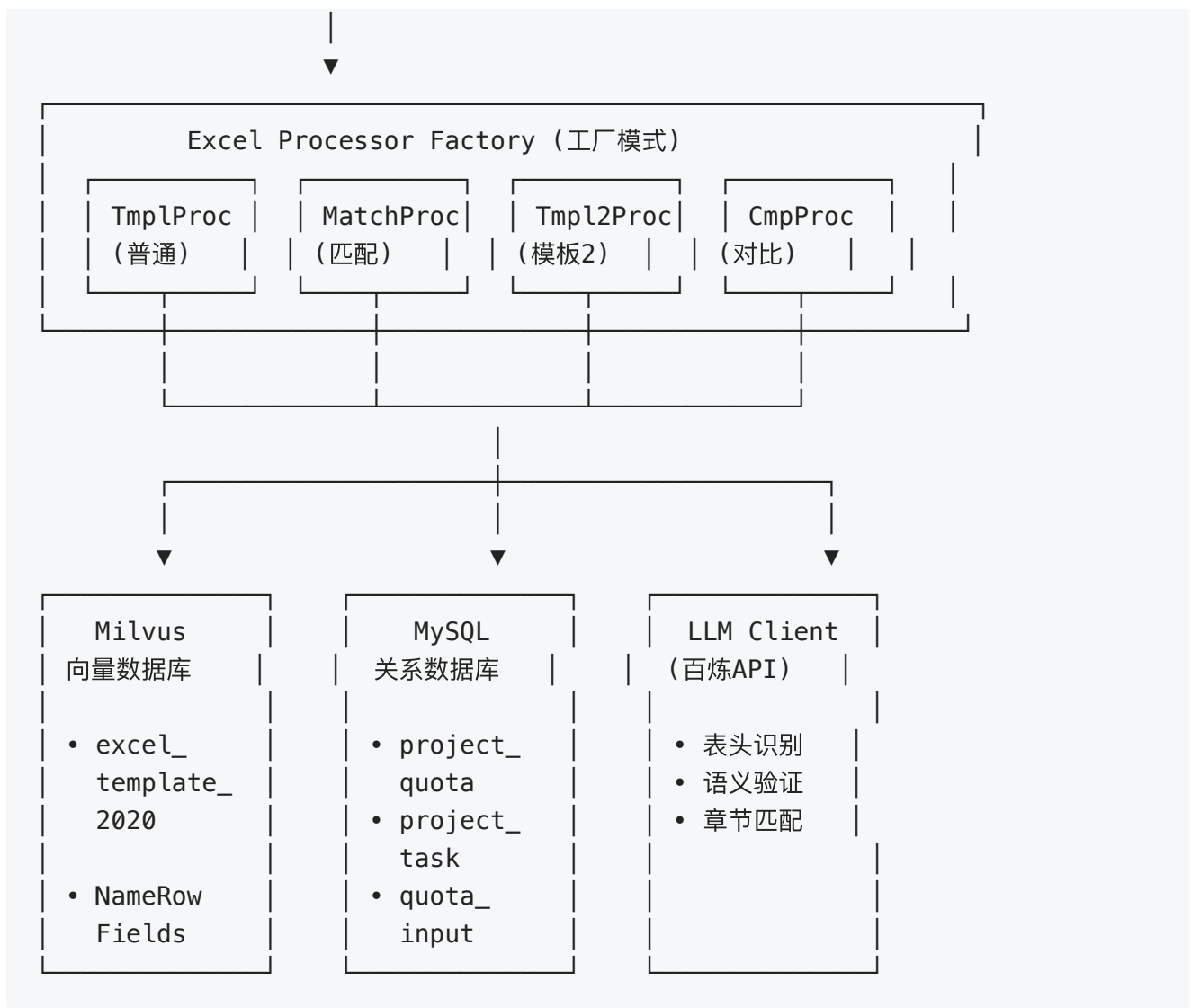
1.3 技术栈

技术分类	技术选型
Web框架	FastAPI 0.115.11
向量数据库	Milvus 2.5.6 (BGE-M3 Embedding)
关系数据库	MySQL (SQLAlchemy 2.0.39)
大语言模型	阿里云百炼API (OpenAI兼容接口)
数据处理	Pandas 2.2.3, NumPy 2.2.4
Excel处理	openpyxl 3.1.5, xlrd 2.0.1
异步框架	asyncio, uvicorn 0.34.0

2. 系统架构设计

2.1 整体架构





2.2 模块划分

2.2.1 API层 (`app/main.py`)

- **职责**：接收HTTP请求，参数验证，响应格式化
- **特点**：使用FastAPI框架，支持异步处理

2.2.2 任务管理层 (`app/apps/task_manager.py`)

- **职责**：
 - 任务提交与调度
 - 任务状态跟踪
 - 项目级并发控制
 - 用户映射保存
- **关键机制**：
 - 项目锁 (`project_locks`)：防止同一项目的任务并发执行
 - 任务存储 (`task_store`)：内存中存储任务状态和结果
 - 运行跟踪 (`running_projects`)：跟踪每个项目正在运行的任务

2.2.3 处理器层 (`app/apps/excel_processors/`)

- **职责**：实现不同业务场景的Excel处理逻辑
- **设计模式**：工厂模式 + 策略模式
- **处理器类型**：
 - TmplExcelProcessor：普通任务（向量匹配）
 - MatchExcelProcessor：匹配任务（数量匹配）
 - Tmpl2ExcelProcessor：模板任务2（模板复制）
 - CmpExcelProcessor：对比任务（版本差异）

2.2.4 工具层 (app/apps/excel_processors/utils.py)

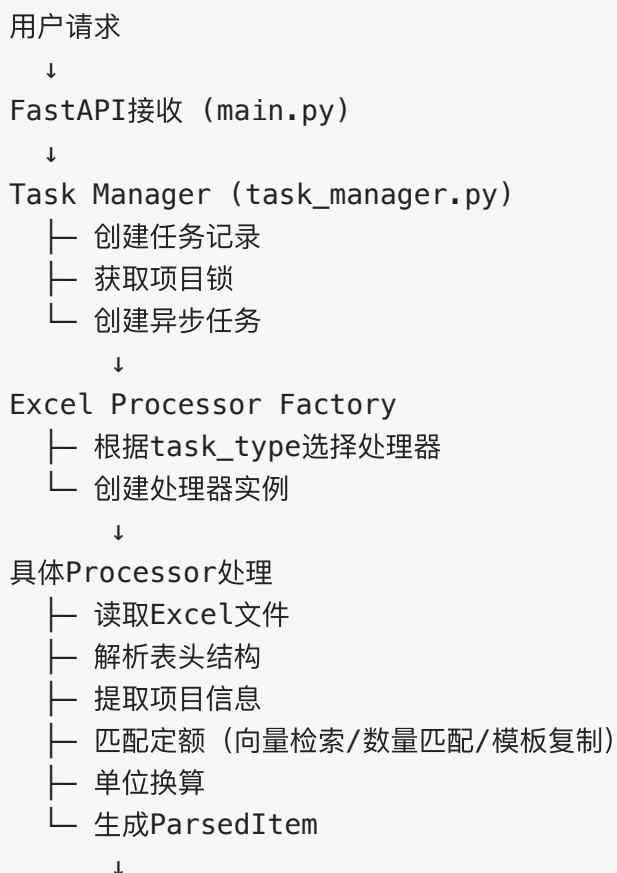
- **职责**：提供通用工具函数
 - Excel表头解析
 - 单位换算
 - 数量计算
 - 向量检索封装

2.2.5 数据模型层

- **Pydantic模型** (app/apps/pydantic_schemas.py)：API请求/响应模型
- **MySQL模型** (app/apps/mysql_schemas.py)：数据库ORM模型
- **Milvus模型** (app/apps/milvus_schemas.py)：向量数据库Schema定义

2.3 数据流设计

2.3.1 任务处理数据流



逐行yield结果

↓

Task Manager更新状态

↓

前端轮询获取结果

2.3.2 用户映射学习数据流

用户在前端确认映射

↓

调用 /api/v1/quota_map

↓

提取 ex_row (Excel行内容)

↓

生成 MD5_ID (唯一标识)

↓

检查Milvus是否已存在

- ├ 已存在 → 返回提示
- └ 不存在 → 继续

↓

BGE-M3向量化

↓

保存到 excel_template_2020 集合

↓

影响后续任务的向量检索

- ├ 普通任务 (TmplExcelProcessor)
- └ 对比任务新增项 (CmpExcelProcessor)

3. 核心模块设计

3.1 任务管理器模块 (task_manager.py)

3.1.1 核心数据结构

```
# 任务存储字典
task_store: Dict[int, Dict[str, Any]] = {
    task_id: {
        "status": TaskResult.RUNNING/SUCCESS/FAILED,
        "data": List[ParsedItem], # 解析结果列表
        "reason": Optional[str], # 失败原因
        "project_id": int,
        "last_sent_index": int, # 上次发送的索引位置
        "total_rows": int, # 总行数
        "processed_rows": int # 已处理行数
    }
}
```

```
# 项目运行跟踪
running_projects: Dict[int, int] = {
    project_id: task_id # 每个项目当前运行的任务ID
}

# 项目锁字典
project_locks: Dict[int, asyncio.Lock] = {
    project_id: asyncio.Lock() # 防止同一项目并发执行
}
```

3.1.2 关键函数设计

submit_task(request: TaskSubmitRequest) -> None

- **功能**：提交任务并初始化任务状态
- **流程**：
 1. 确保项目锁存在
 2. 初始化任务状态字典
 3. 创建异步处理任务

_process_task(request: TaskSubmitRequest) -> None

- **功能**：异步处理任务的核心逻辑
- **流程**：
 1. 获取项目锁（防止并发）
 2. 检查是否有相同项目的任务正在运行
 3. 根据 task_type 判断任务类型
 4. 创建对应的处理器实例
 5. 异步迭代处理Excel，逐行yield结果
 6. 更新任务状态（成功/失败）

get_task_status(task_id: int) -> Optional[Dict[str, Any]]

- **功能**：获取任务状态
- **返回**：任务状态字典，包含状态、数据、进度等信息

cancel_task(task_id: int) -> bool

- **功能**：取消正在运行的任务
- **流程**：
 1. 检查任务是否存在
 2. 清除运行标记
 3. 更新任务状态为FAILED

save_quota_mapping_to_milvus(...) -> tuple[bool, str]

- **功能**：保存用户确认的映射关系到Milvus
- **流程**：
 1. 生成MD5_ID（基于ex_row）
 2. 检查是否已存在
 3. BGE-M3向量化
 4. 保存到 excel_template_2020 集合

3.2 Excel处理器模块

3.2.1 基类设计 (base_processor.py)

```
class BaseExcelProcessor:
    """Excel处理器的基类，定义处理接口"""

    async def process_excel(self, request: TaskSubmitRequest):
        """
        处理Excel文件的主方法

        Yields:
            ParsedItem: 每处理完一行就立即生成一个解析结果
        """
        raise NotImplementedError("子类必须实现process_excel方法")
```

设计特点：

- 使用异步生成器模式，逐行yield结果，实现流式处理
- 支持大文件处理，避免内存溢出

3.2.2 工厂模式 (__init__.py)

```
class ExcelProcessorFactory:
    @staticmethod
    def create_processor(
        processor_type: str,
        llm_client: AsyncOpenAI,
        milvus_client: MilvusClient,
        bge_m3_ef: BGEM3EmbeddingFunction
    ) -> BaseExcelProcessor:
        """根据processor_type创建对应的处理器实例"""
```

支持的处理器类型：

- "tmpl" → TmplExcelProcessor（普通任务）
- "match" → MatchExcelProcessor（匹配任务）
- "tmpl2" → Tmpl2ExcelProcessor（模板任务2）
- "cmp" → CmpExcelProcessor（对比任务）

- "llm" → LLMExcelProcessor (LLM处理, 已废弃)

3.2.3 普通任务处理器 (`tpl_processor.py`)

核心流程：

1. 表头识别
 - 0305章节：使用预定义JSON模板
 - 其他章节：LLM动态识别
2. 桥梁名称提取：使用LLM从表头提取桥梁名称
3. 章节匹配：根据桥梁名称匹配具体的章节代码
4. 逐行处理：
 - Excel行内容 → BGE-M3向量化
 - Milvus向量检索 (相似度 ≥ 0.92)
 - 根据MD5_ID从MySQL查询定额模板
 - 展开定额 (处理特殊定额编号如HT开头)
 - 单位换算和数量计算
 - 章节代码映射

关键技术：

- 向量相似度检索 (BGE-M3 + Milvus)
- 模板展开机制 (`need_expand` 标志)
- 章节代码映射算法

3.2.4 匹配任务处理器 (`match_processor.py`)

核心流程：四轮渐进式匹配策略

第一轮：精确匹配

- 条件： `amount == project_quantity` (保留3位小数)
- 单位：字符串完全一致

第二轮：单位等价匹配

- 单位标准化映射 ("平方米"→"m2")
- 系数提取 ("100m2"→系数100+单位"m2")
- 数量换算后比较

第三轮：LLM语义验证

- 忽略单位名称差异
- 使用LLM判断项目名称语义相似度
- 置信度阈值 ≥ 0.7

第四轮：冲突解决

- 多对一冲突：使用LLM选择最佳匹配
- 一对多关系：全部保留

3.2.5 模板任务2处理器 (`tmpl2_processor.py`)

核心流程：

1. **加载模板定额**：从 `parent_id` 指定的父任务读取所有定额输入
2. **构建`ex_row`索引**：

```
ex_row_index = {  
    "基础 | 围堰及防护 | C30混凝土": [quota1, quota2, ...],  
    "C30混凝土": [quota3, ...], # 同时索引最后一部分  
}
```

3. **Excel行匹配**：
 - 直接匹配：Excel行内容与`ex_row`完全一致
 - 部分匹配：Excel行内容包含在`ex_row`中，或反之（长度>3）
4. **章节代码转换**：

```
模板item_code: 0302-01-05-02-01-02-01  
选中章节code: 0302-01-05-03-01
```

计算过程：

1. 选中章节长度：5个部分
2. 取模板后缀：-02-01（从第5个部分之后）
3. 新`item_code`：0302-01-05-03-01-02-01

5. **定额信息复制**：复制定额编号、调整说明等，重新计算数量

3.2.6 对比任务处理器 (`cmp_processor.py`)

核心流程：

1. **文件识别**：识别新旧版本文件（`is_new` 标志）
2. **并行处理**：使用 `ThreadPoolExecutor` 并行处理新旧文件
3. **差异分类**：
 - **新增项** (`new_only`)：仅在新文件中出现且数量>0
 - **删除项** (`old_only`)：仅在旧文件中出现且数量>0
 - **修改项** (`different`)：两个文件都有但数量不同
 - **不变项** (`matched`)：两个文件都有且数量相同
4. **定额查找**：从父任务定额中按 `ex_row` 查找匹配
5. **处理策略**：
 - 修改项/删除项：复用父任务定额，更新数量
 - 新增项：调用 `process_single_row_0305` 进行向量匹配

4. 数据模型设计

4.1 Pydantic数据模型 (pydantic_schemas.py)

4.1.1 任务相关模型

TaskSubmitRequest : 任务提交请求

```
class TaskSubmitRequest(BaseModel):
    task_id: int                # 任务ID
    parent_id: int              # 父任务ID (模板任务2和对比任务需要)
    version: str                 # 版本号
    project_id: str              # 项目ID
    project_name: str            # 项目名称
    project_stage: str           # 项目阶段 (初设/施工图等)
    selected_zgs_id: int         # 选中的总概算ID
    task_type: int               # 任务类型 (0-普通, 1-匹配, 2-模板2)
    zgs_list: List[ZGSItem]     # 总概算列表
    selected_chapter: HierarchyItem # 选中的章节
    files: List[ExcelFile]      # Excel文件列表
```

TaskResponse : 任务状态响应

```
class TaskResponse(BaseModel):
    task_id: int                # 任务ID
    result: int                  # 状态 (-1失败, 0运行中, 1成功)
    data: List[ParsedItem]      # 解析结果列表
    reason: Optional[str]       # 失败原因
    total_rows: int              # 总行数
    processed_rows: int          # 已处理行数
```

ParsedItem : 解析结果项

```
class ParsedItem(BaseModel):
    zgs_id: int                  # 总概算ID
    zgs_code: str                # 总概算代码
    item_id: int                 # 条目ID
    item_code: str               # 条目代码
    dinge_code: str              # 定额编号
    dinge_adjust: str            # 定额调整
    entry_name: str              # 工程或费用项目名称
    units: str                   # 单位
    amount: float                # 数量
    target_id: int               # 目标ID
    file_id: str                 # 文件ID
    ex_cell: str                 # Excel单元格位置
```

```
ex_row: str          # Excel行内容
ex_unit: str          # Excel单位
ex_amount: float      # Excel数量
note: str             # 说明
```

4.1.2 用户映射模型

QuotaMapRequest：定额映射请求

```
class QuotaMapRequest(BaseModel):
    input_id: int          # 项目定额条目ID
    quota_id: int          # 定额输入ID
    item_code: str         # 章节编号（完整）
    root_chapter_code: str # 根章节代码
    quota_code: str        # 定额编号
    quota_name: str        # 定额名称
    entry_name: str        # 工程或费用项目名称
    units: str             # 单位
    work_content: Optional[str] # 工作内容
    chapter_path: Optional[str] # 章节路径
    ex_row: Optional[str]   # Excel原始行内容
    quota_adjustment: Optional[str] # 定额调整
```

4.2 MySQL数据模型 (mysql_schemas.py)

4.2.1 项目任务表 (ProjectTaskModel)

表名： project_task

主要字段：

- id：主键
- parent_id：父任务ID
- task_name：任务名称
- task_type：任务类型（0-普通，1-匹配，2-模板）
- project_id：项目编号
- item_id：条目序号
- item_code：条目编号
- process_status：处理状态（0-未处理，1-处理中，2-已处理，3-处理失败）

4.2.2 项目定额表 (ProjectQuotaModel)

表名： project_quota

主要字段：

- id：主键

- `task_id`：任务编号
- `project_id`：项目编号
- `item_id`：条目序号
- `item_code`：条目编号
- `quota_code`：定额编号
- `quota_adjustment`：定额调整
- `entry_name`：工程或费用项目名称
- `units`：单位
- `amount`：数量
- `ex_file`：Excel文件
- `ex_cell`：数量单元格位置
- `ex_row`：该行内容（由逗号连接多个单元格）
- `ex_unit`：Excel中给出的单位
- `ex_amount`：Excel中给出的数量
- `note`：说明

4.2.3 定额模板表 (`DingeTemplateModel`)

表名： `xparser_dinge_template`

主要字段：

- `id`：主键
- `chapter`：章节
- `md5_id`：MD5 ID（用于关联Milvus中的模板）
- `name_str`：名称字符串
- `dinge_code`：定额编号
- `dinge_name`：定额名称
- `dinge_adjust`：定额调整
- `dinge_unit`：单位
- `conditional`：是否条件定额
- `note`：备注

索引： `idx_md5_id` (`md5_id`字段)

4.2.4 定额库表 (`QuotaLibraryModel2020`)

表名： `xparser_quota_library_2020`

主要字段：

- `id`：主键
- `book_code`：书号
- `quota_code`：定额编号

- `quota_name` : 定额名称
- `unit` : 单位
- `work_content` : 工作内容
- `base_price` : 基价
- `labor_cost` : 工费
- `material_cost` : 料费
- `machine_cost` : 机费

索引: `idx_quota_code` (`quota_code`字段)

4.3 Milvus向量数据库模型 (`milvus_schemas.py`)

4.3.1 Excel模板集合 (`excel_template_2020`)

集合名称: `excel_template_2020`

Schema定义: `NameRowFields`

字段结构:

```
{
  "id": INT64 (主键, 自增),
  "project_stage": VARCHAR(32),           # 项目阶段 (如"初设")
  "root_chapter_code": VARCHAR(32),       # 根章节代码 (如"0305")
  "template_ver": INT8,                   # 模板版本
  "md5_id": VARCHAR(32),                  # MD5 ID (基于ex_row)
  "need_expand": BOOL,                    # 是否需要扩展
  "item_name": VARCHAR(512),              # 细目名称 (Excel行内容)
  "chapter_template": VARCHAR(64),        # 章节模板
  "main_vector": FLOAT_VECTOR(1024)      # 主向量 (BGE-M3生成)
}
```

用途:

- 存储Excel行模板的向量表示
- 支持向量相似度检索
- 包含系统模板和用户确认的映射

检索方式:

- 使用 `main_vector` 字段进行向量相似度检索
- 相似度阈值: ≥ 0.92
- 支持按 `root_chapter_code` 过滤

5. API接口设计

5.1 任务提交接口

接口路径： `POST /api/v1/task_submit`

请求体： `TaskSubmitRequest`

响应： `TaskResponse`

功能说明：

- 提交Excel处理任务
- 立即返回，任务异步执行
- 返回任务ID和初始状态（RUNNING）

示例请求：

```
{
  "task_id": 12345,
  "parent_id": 0,
  "version": "1.0",
  "project_id": "PROJ001",
  "project_name": "XX特大桥",
  "project_stage": "初设",
  "selected_zgs_id": 1,
  "task_type": 0,
  "zgs_list": [...],
  "selected_chapter": {
    "item_id": 100,
    "item_code": "0305-01-01-01",
    "item_name": "XX特大桥"
  },
  "files": [
    {
      "file_id": "file001",
      "file_type": "xlsx",
      "content": "base64编码的文件内容",
      "is_new": 1
    }
  ]
}
```

示例响应：

```
{
  "task_id": 12345,
  "result": 0,
  "data": [],
  "reason": "运行中",
}
```

```
"total_rows": 0,
"processed_rows": 0
}
```

5.2 任务状态查询接口

接口路径： GET /api/v1/task_status 或 POST /api/v1/task_status

查询参数： task_id: int

响应： TaskResponse

功能说明：

- 查询任务处理状态和结果
- 支持增量获取（只返回新数据）
- 使用 last_sent_index 跟踪已发送的数据位置

状态值说明：

- -1：失败（FAILED）
- 0：运行中（RUNNING）
- 1：成功（SUCCESS）

示例响应：

```
{
  "task_id": 12345,
  "result": 0,
  "data": [
    {
      "zgs_id": 1,
      "zgs_code": "ZGS001",
      "item_id": 100,
      "item_code": "0305-01-01-01-02-01",
      "dinge_code": "QY-51-01-02",
      "dinge_adjust": "",
      "entry_name": "C30混凝土",
      "units": "m3",
      "amount": 125.5,
      "target_id": 0,
      "file_id": "file001",
      "ex_cell": "C17",
      "ex_row": "基础 | 围堰及防护 | C30混凝土",
      "ex_unit": "立方米",
      "ex_amount": 125.5,
      "note": ""
    }
  ],
}
```

```
"reason": null,  
"total_rows": 50,  
"processed_rows": 10  
}
```

5.3 任务取消接口

接口路径： GET /api/v1/cancel_task 或 POST /api/v1/cancel_task

查询参数： task_id: int

响应： TaskResponse

功能说明：

- 取消正在运行的任务
- 清除项目运行标记
- 更新任务状态为FAILED

5.4 定额映射保存接口

接口路径： POST /api/v1/quota_map

请求体： QuotaMapRequest

响应： QuotaMapResponse

功能说明：

- 保存用户确认的Excel行与定额的映射关系
- 将映射保存到Milvus向量数据库
- 影响后续任务的向量检索结果

示例请求：

```
{  
  "input_id": 12345,  
  "quota_id": 67890,  
  "item_code": "0305-01-01-01-02-01",  
  "root_chapter_code": "0305",  
  "quota_code": "QY-51-01-02",  
  "quota_name": "C30混凝土",  
  "entry_name": "C30混凝土",  
  "units": "m3",  
  "work_content": "",  
  "chapter_path": "",  
  "ex_row": "基础 | 围堰及防护 | C30混凝土",  
  "quota_adjustment": ""  
}
```


示例响应：

```
{
  "success": true,
  "message": "Excel 行模板已成功保存到向量数据库",
  "data": {
    "input_id": 12345,
    "quota_id": 67890,
    "quota_code": "QY-51-01-02",
    "root_chapter_code": "0305",
    "ex_row": "基础 | 围堰及防护 | C30混凝土"
  }
}
```

6. 任务处理流程设计

6.1 任务类型判断逻辑

系统在 `task_manager.py` 的 `_process_task` 函数中根据以下规则判断任务类型：

```
if request.task_type == 1:
    → 匹配任务 (MatchExcelProcessor)

elif request.task_type == 2:
    → 模板任务2 (Tpl2ExcelProcessor)

else:
    if len(request.files) > 1 且存在 is_new==0 的文件:
        → 对比任务 (CmpExcelProcessor)
    else:
        → 普通任务 (TplExcelProcessor)
```

6.2 普通任务处理流程

1. 接收任务请求
↓
2. 获取项目锁（防止并发）
↓
3. 创建TplExcelProcessor实例
↓
4. 读取Excel文件（base64解码）
↓
5. 表头识别
 - └ 0305章节：使用JSON模板
 - └ 其他章节：LLM动态识别

- ↓
6. 提取桥梁名称 (LLM)
 - ↓
 7. 匹配章节代码
 - ↓
 8. 逐行处理Excel
 - ├ 提取行内容 (ex_row)
 - ├ BGE-M3向量化
 - ├ Milvus向量检索 (相似度 ≥ 0.92)
 - ├ 根据MD5_ID查询MySQL模板
 - ├ 展开定额 (处理need_expand)
 - ├ 单位换算
 - ├ 数量计算
 - ├ 章节代码映射
 - └ yield ParsedItem
 - ↓
 9. 更新任务状态为SUCCESS

6.3 匹配任务处理流程

1. 接收任务请求
- ↓
2. 获取项目锁
- ↓
3. 创建MatchExcelProcessor实例
- ↓
4. 从MySQL加载定额输入记录
- ↓
5. 读取Excel文件
- ↓
6. 提取Excel行数据
- ↓
7. 四轮匹配策略
 - ├ 第一轮：精确匹配 (数量+单位完全一致)
 - ├ 第二轮：单位等价匹配 (单位标准化+数量换算)
 - ├ 第三轮：LLM语义验证 (忽略单位，判断语义相似度)
 - └ 第四轮：冲突解决 (多对一使用LLM选择最佳)
- ↓
8. 生成匹配结果
 - ├ 已匹配：Excel行 ↔ 定额记录
 - ├ 未匹配 (Excel端)：Excel中有但定额中无
 - └ 未匹配 (定额端)：定额中有但Excel中无
- ↓
9. yield匹配结果

6.4 模板任务2处理流程

1. 接收任务请求 (包含parent_id)
↓
2. 获取项目锁
↓
3. 创建Tpl2ExcelProcessor实例
↓
4. 从MySQL加载父任务的定额输入记录
↓
5. 构建ex_row索引
{
 "完整ex_row": [quota1, quota2, ...],
 "最后一部分": [quota3, ...]
}
↓
6. 读取新Excel文件
↓
7. 逐行匹配模板定额
 - ├ 直接匹配: Excel行 == ex_row
 - ├ 部分匹配: Excel行包含在ex_row中, 或反之
 - └ 未匹配: 找不到对应模板
↓
8. 章节代码转换
 - ├ 提取选中章节代码长度
 - ├ 提取模板章节代码后缀
 - └ 拼接新章节代码
↓
9. 复制定额信息
 - ├ 复制: quota_code, quota_adjustment, entry_name, units
 - ├ 调整: item_id, item_code
 - └ 重算: amount (基于新Excel的数量和单位)
↓
10. yield ParsedItem

6.5 对比任务处理流程

1. 接收任务请求 (包含2个文件, is_new标识)
↓
2. 获取项目锁
↓
3. 创建CmpExcelProcessor实例
↓
4. 并行处理新旧文件
 - ├ 线程1: 处理新文件 → new_file_result
 - └ 线程2: 处理旧文件 → old_file_result
↓
5. 差异对比分类
 - ├ 新增项 (new_only): 仅在新文件中出现
 - └ 删除项 (old_only): 仅在旧文件中出现

```
└─ 修改项 (different): 两个文件都有但数量不同
└─ 不变项 (matched): 两个文件都有且数量相同
↓
6. 从父任务加载定额索引 (按ex_row)
↓
7. 处理各类差异项
└─ 修改项: 查找父任务定额 → 更新数量
└─ 删除项: 查找父任务定额 → 数量置0
└─ 新增项: 调用process_single_row_0305 → 向量匹配
↓
8. yield处理结果
```

6.6 并发控制机制

项目级锁机制:

- 每个项目维护一个 `asyncio.Lock`
- 同一项目的任务串行执行，防止数据冲突
- 不同项目的任务可以并行执行

实现代码:

```
# 确保项目锁存在
if project_id not in project_locks:
    project_locks[project_id] = asyncio.Lock()

# 获取项目锁
async with project_locks[project_id]:
    # 检查是否有相同项目的任务正在运行
    if project_id in running_projects:
        # 标记任务为失败
        return

    # 标记项目正在运行该任务
    running_projects[project_id] = task_id

    try:
        # 处理任务
        ...
    finally:
        # 清除运行标记
        if running_projects.get(project_id) == task_id:
            del running_projects[project_id]
```

7. 关键技术实现

7.1 向量相似度检索

7.1.1 向量化模型

模型： BGE-M3 (BAAI/bge-m3)

- **维度：** 1024
- **设备：** CPU
- **精度：** FP32 (CPU不支持FP16)

向量化代码：

```
from pymilvus.model.hybrid import BGEM3EmbeddingFunction

bge_m3_ef = BGEM3EmbeddingFunction(
    model_name='BAAI/bge-m3',
    device='cpu',
    use_fp16=False
)

# 向量化文档
docs_embeddings = bge_m3_ef.encode_documents([ex_row])
main_vector = docs_embeddings['dense'][0].tolist()
```

7.1.2 Milvus检索

检索参数：

- **相似度阈值：** ≥ 0.92
- **返回数量：** top 10
- **过滤条件：** `root_chapter_code == "0305"` (可选)

检索代码：

```
search_params = {
    "metric_type": "IP", # 内积
    "params": {"nprobe": 10}
}

results = milvus_client.search(
    collection_name="excel_template_2020",
    data=[main_vector],
    anns_field="main_vector",
    search_params=search_params,
    limit=10,
    expr=f'{NameRowFields.ROOT_CHAPTER_CODE} == "{root_chapter_code}"',
    output_fields=[NameRowFields.MD5_ID, NameRowFields.ITEM_NAME, ...]
)
```

7.2 单位换算机制

7.2.1 单位标准化映射

```
UNIT_MAPPING = {  
    "米": "m",  
    "平方米": "m2", "m²": "m2", "m^2": "m2",  
    "立方米": "m3", "m³": "m3", "m^3": "m3",  
    "吨": "t",  
    "千克": "kg",  
    "公里": "km",  
    ...  
}
```

7.2.2 系数提取

支持带系数的单位，如"100m2"：

- 提取系数：100
- 基础单位：m2
- 数量换算：`amount = excel_amount * coefficient`

7.2.3 数量计算

```
def calculate_amount(  
    excel_amount: float,  
    excel_unit: str,  
    quota_unit: str  
) -> float:  
    """  
    计算定额数量  
  
    1. 标准化单位  
    2. 提取系数  
    3. 单位换算  
    4. 计算最终数量  
    """  
    # 标准化Excel单位  
    excel_unit_normalized = normalize_unit(excel_unit)  
  
    # 提取系数  
    coefficient, base_unit = extract_coefficient(excel_unit_normalized)  
  
    # 单位换算（如果需要）  
    if base_unit != quota_unit:  
        conversion_factor = get_conversion_factor(base_unit, quota_unit)  
        return excel_amount * coefficient * conversion_factor
```

```
return excel_amount * coefficient
```

7.3 LLM集成

7.3.1 客户端配置

```
from openai import AsyncOpenAI

llm_client = AsyncOpenAI(
    api_key="sk-xxx",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)
```

7.3.2 主要应用场景

1. 表头结构识别（非0305章节）

- **输入**：Excel表头行
- **输出**：JSON格式的表头结构（项目名称列、单位列、数量列等）

2. 桥梁名称提取

- **输入**：Excel表头内容
- **输出**：桥梁名称（如"XX特大桥"）

3. 章节匹配

- **输入**：桥梁名称 + 章节列表
- **输出**：匹配的章节代码

4. 语义相似度验证（匹配任务第三轮）

- **输入**：Excel项目名称 + 定额项目名称
- **输出**：{"is_match": true, "confidence": 0.95, "reason": "..."}

7.3.3 提示词管理

提示词存储在 app/apps/prompts.py 中，支持：

- 表头识别提示词
- 桥梁名称提取提示词
- 章节匹配提示词
- 语义验证提示词

7.4 模板展开机制

7.4.1 need_expand标志

在Milvus的 `excel_template_2020` 集合中, `need_expand` 字段标识模板是否需要展开:

- `need_expand = True`: 扩展模板
 - Excel行数量作为系数
 - 定额数量 = 模板数量 × Excel数量
- `need_expand = False`: 非扩展模板
 - 直接使用Excel数量 (经过单位换算)

7.4.2 特殊定额处理

HT开头混凝土定额:

- 需要根据标号和运距调整
- 在 `note` 字段中标注"需根据标号和运距调整"

条件定额:

- `conditional = True` 的定额需要特殊处理
- 可能需要额外的条件判断

7.5 章节代码映射算法

7.5.1 模板任务2的章节代码转换

输入:

- 模板 `item_code`: `0302-01-05-02-01-02-01`
- 选中章节 `item_code`: `0302-01-05-03-01`

计算过程:

1. 选中章节长度: 5个部分 (`0302-01-05-03-01`)
2. 提取模板后缀: 从第5个部分之后的部分 (`-02-01`)
3. 拼接新章节代码: `0302-01-05-03-01-02-01`

代码实现:

```
def map_chapter_code(
    template_item_code: str,
    target_item_code: str
) -> str:
    """章节代码映射"""
    template_parts = template_item_code.split('-')
    target_parts = target_item_code.split('-')

    # 目标章节长度
    target_length = len(target_parts)
```



```
# 提取模板后缀（从目标长度之后的部分）
template_suffix = '-'.join(template_parts[target_length:])

# 拼接新章节代码
if template_suffix:
    return f'{target_item_code}-{template_suffix}'
else:
    return target_item_code
```

8. 部署与配置

8.1 环境要求

Python版本： Python 3.8+

系统要求：

- Linux / macOS / Windows
- 至少4GB内存
- 支持Docker（推荐）

8.2 依赖安装

requirements.txt：

```
fastapi==0.115.11
uvicorn==0.34.0
pydantic==2.10.6
pymilvus==2.5.6
pymilvus.model==0.3.2
openai==1.70.0
SQLAlchemy==2.0.39
PyMySQL==1.1.1
pandas==2.2.3
openpyxl==3.1.5
xlrd==2.0.1
numpy==2.2.4
FlagEmbedding==1.3.4
torch==2.6.0
```

安装命令：

```
pip install -r requirements.txt
```

8.3 配置文件

8.3.1 环境变量配置 (config/prod.env)

```
# MySQL数据库配置
MYSQL_HOST=mysql
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=password
MYSQL_DATABASE=railway_costing

# Milvus配置
MILVUS_URI=http://milvus:19530
MILVUS_TOKEN=root:Milvus
MILVUS_DB_NAME=ztshy2020

# LLM配置 (百炼API)
OPENAI_API_KEY=sk-xxx
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1

# 服务配置
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
```

8.3.2 代码中的配置

task_manager.py:

```
llm_client = AsyncOpenAI(
    api_key="sk-xxx",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

milvus_client = MilvusClient(
    uri="http://milvus:19530",
    token="root:Milvus"
)
milvus_client.use_database(db_name="ztshy2020")

bge_m3_ef = BGEM3EmbeddingFunction(
    model_name='BAAI/bge-m3',
    device='cpu',
    use_fp16=False
)
```

8.4 Docker部署

8.4.1 Dockerfile

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.4.2 Docker Compose配置

```
version: '3.8'

services:
  xparser:
    build: .
    ports:
      - "8000:8000"
    environment:
      - MYSQL_HOST=mysql
      - MILVUS_URI=http://milvus:19530
    depends_on:
      - mysql
      - milvus
    volumes:
      - ./app:/app/app

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: railway_costing
    volumes:
      - mysql_data:/var/lib/mysql

  milvus:
    image: milvusdb/milvus:latest
    ports:
      - "19530:19530"
    volumes:
      - milvus_data:/var/lib/milvus

volumes:
```

```
mysql_data:
milvus_data:
```

8.5 启动服务

开发模式：

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

生产模式：

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

9. 性能与安全

9.1 性能优化

9.1.1 异步处理

- **异步任务**：使用 `asyncio.create_task` 创建异步任务
- **异步生成器**：处理器使用 `async def` 和 `yield` 实现流式处理
- **异步数据库操作**：使用SQLAlchemy的异步引擎（如需要）

9.1.2 批量处理优化

- **向量检索批量**：支持批量向量检索（如需要）
- **数据库批量插入**：使用批量插入减少数据库交互

9.1.3 内存管理

- **流式处理**：逐行yield结果，避免一次性加载所有数据
- **文件处理**：使用Pandas的chunk读取大文件（如需要）

9.1.4 缓存机制

- **项目锁缓存**： `project_locks` 字典缓存项目锁
- **任务状态缓存**： `task_store` 字典缓存任务状态

9.2 安全考虑

9.2.1 输入验证

- **Pydantic验证**：所有API请求使用Pydantic模型验证
- **文件类型检查**：验证文件类型和大小
- **Base64解码**：安全解码Base64编码的文件内容

9.2.2 并发控制

- **项目级锁**：防止同一项目的任务并发执行
- **任务状态检查**：检查任务是否已存在或正在运行

9.2.3 错误处理

- **异常捕获**：所有关键操作都有异常处理
- **错误日志**：记录详细的错误信息
- **任务状态更新**：失败时更新任务状态和原因

9.2.4 数据安全

- **MD5去重**：使用MD5_ID防止重复数据
- **数据验证**：验证数据完整性和有效性

9.3 监控与日志

9.3.1 日志配置

```
import logging

logger = logging.getLogger('uvicorn.error')

# 记录关键操作
logger.info(f"任务 {task_id} 开始处理")
logger.error(f"任务 {task_id} 处理失败: {str(e)}")
```

9.3.2 监控指标

- **任务处理时间**：记录每个任务的处理时间
- **任务成功率**：统计任务成功/失败率
- **处理行数**：跟踪处理进度

10. 扩展性设计

10.1 新增处理器类型

扩展步骤：

1. 创建新的处理器类，继承 BaseExcelProcessor
2. 实现 process_excel 方法
3. 在 ExcelProcessorFactory 中注册新处理器
4. 在 task_manager.py 中添加任务类型判断逻辑

示例：

```
# 1. 创建新处理器
class CustomExcelProcessor(BaseExcelProcessor):
    async def process_excel(self, request: TaskSubmitRequest):
        # 实现处理逻辑
        ...

# 2. 注册到工厂
processors = {
    ...
    "custom": lambda: CustomExcelProcessor(...),
}

# 3. 添加任务类型判断
if request.task_type == 3:
    processor_type = "custom"
```

10.2 新增数据源

支持新的数据库：

1. 创建新的Schema文件（如 postgresql_schemas.py）
2. 实现数据库连接和查询逻辑
3. 在处理器中使用新的数据源

10.3 新增向量检索策略

扩展检索方式：

1. 实现新的检索函数
2. 在处理器中调用新的检索函数
3. 支持多种检索策略的组合

10.4 插件化设计

未来可扩展为插件系统：

- 处理器插件：动态加载处理器
- 工具插件：动态加载工具函数
- 数据源插件：动态加载数据源

附录

A. 代码文件结构

```

xparser/
├── app/
│   ├── main.py                # FastAPI应用入口
│   └── apps/
│       ├── task_manager.py    # 任务管理器
│       ├── pydantic_schemas.py # Pydantic数据模型
│       ├── mysql_schemas.py  # MySQL数据模型
│       ├── milvus_schemas.py  # Milvus数据模型
│       ├── prompts.py         # LLM提示词
│       └── excel_processors/
│           ├── __init__.py     # 工厂类
│           ├── base_processor.py # 基类
│           ├── tpl_processor.py # 普通任务处理器
│           ├── match_processor.py # 匹配任务处理器
│           ├── tpl2_processor.py # 模板任务2处理器
│           ├── cmp_processor.py # 对比任务处理器
│           ├── llm_processor.py # LLM处理器（已废弃）
│           └── utils.py        # 工具函数
├── config/
│   └── prod.env                # 环境变量配置
├── requirements.txt            # Python依赖
├── Dockerfile                  # Docker镜像定义
└── README.md                   # 项目说明

```

B. 数据库表结构

主要表：

- `project_task`：项目任务表
- `project_quota`：项目定额表
- `xparser_dinge_template`：定额模板表
- `xparser_quota_library_2020`：定额库表

Milvus集合：

- `excel_template_2020`：Excel模板向量集合

C. API接口汇总

接口路径	方法	功能
<code>/api/v1/task_submit</code>	POST	提交任务
<code>/api/v1/task_status</code>	GET/POST	查询任务状态
<code>/api/v1/cancel_task</code>	GET/POST	取消任务
<code>/api/v1/quota_map</code>	POST	保存用户映射

D. 任务类型对照表

task_type	处理器类型	说明
0	TmplExcelProcessor	普通任务（向量匹配）
1	MatchExcelProcessor	匹配任务（数量匹配）
2	Tmpl2ExcelProcessor	模板任务2（模板复制）
其他	CmpExcelProcessor	对比任务（版本差异）

E. 常见问题

Q1：如何处理大文件？

A：系统使用流式处理，逐行yield结果，避免一次性加载所有数据。

Q2：任务失败如何处理？

A：任务失败时会更新状态为FAILED，并记录失败原因。可以重新提交任务。

Q3：如何提高匹配准确率？

A：使用用户映射功能，系统会学习用户确认的映射关系，逐步提高准确率。

Q4：支持哪些Excel格式？

A：支持 .xlsx 和 .xls 格式，使用openpyxl和xlrd库处理。

版本历史

版本	日期	说明
v1.0	2025-08	初始版本，包含四种任务类型和用户映射功能